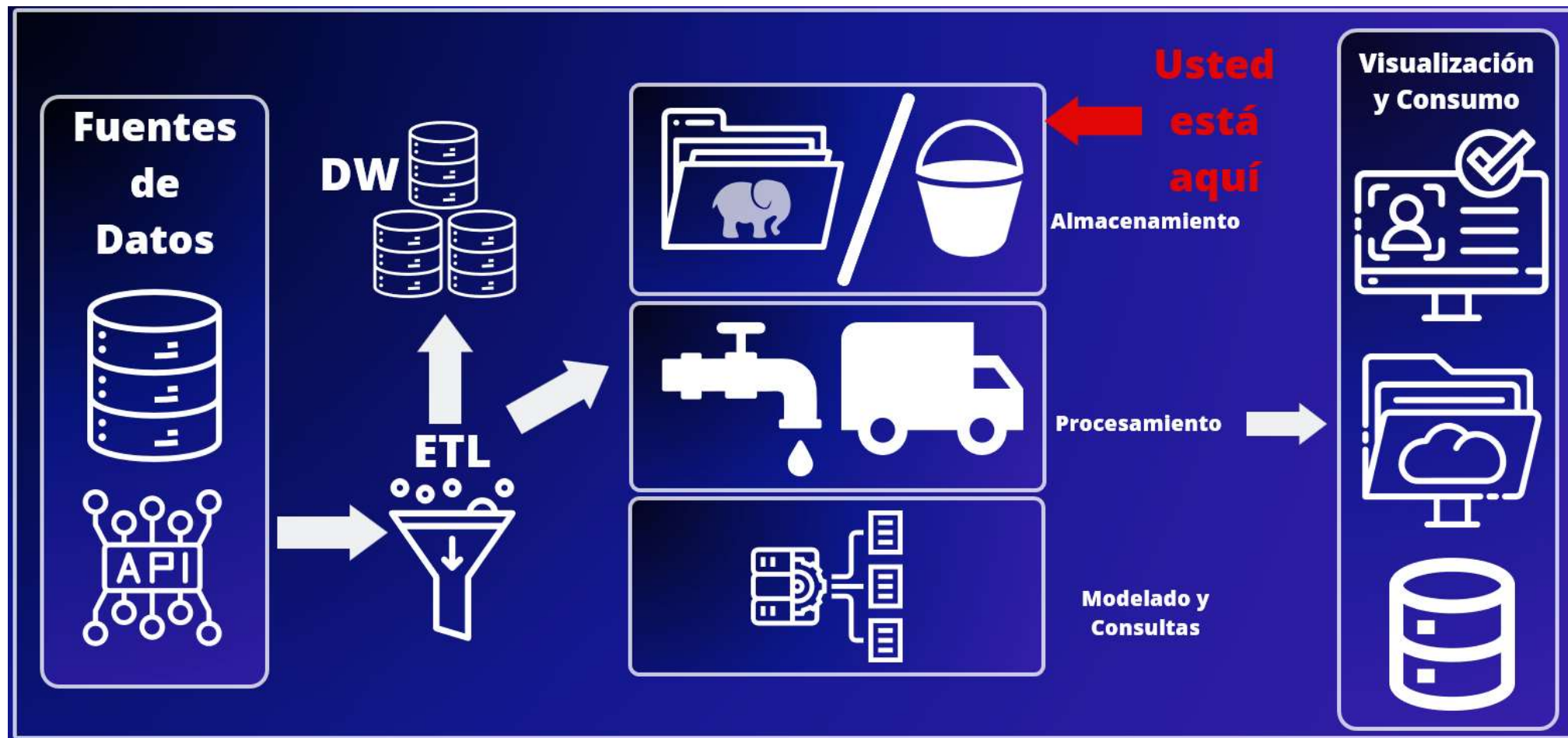




HERRAMIENTAS DE SOFTWARE PARA BIG DATA

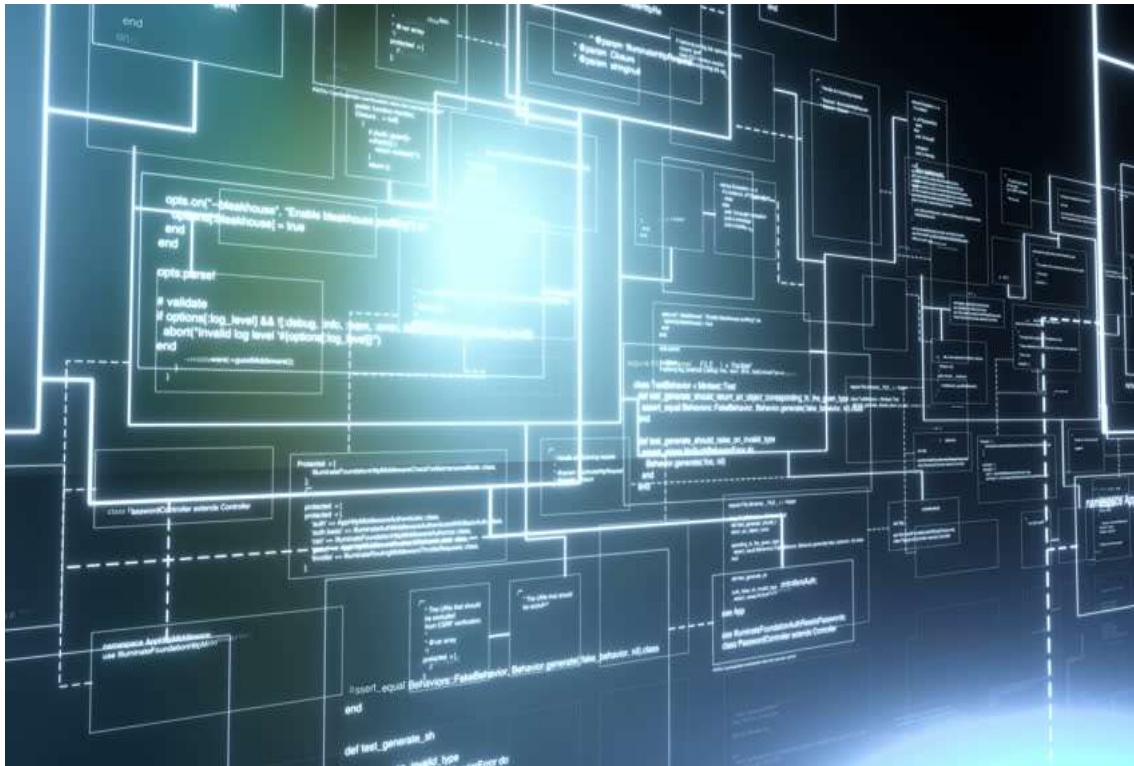
SISTEMAS DE ARCHIVOS DISTRIBUIDOS Y BUCKETS - 04



COMPONENTES DE UN DATALAKE



¿QUÉ ES UN SISTEMA DE ARCHIVOS DISTRIBUIDO?



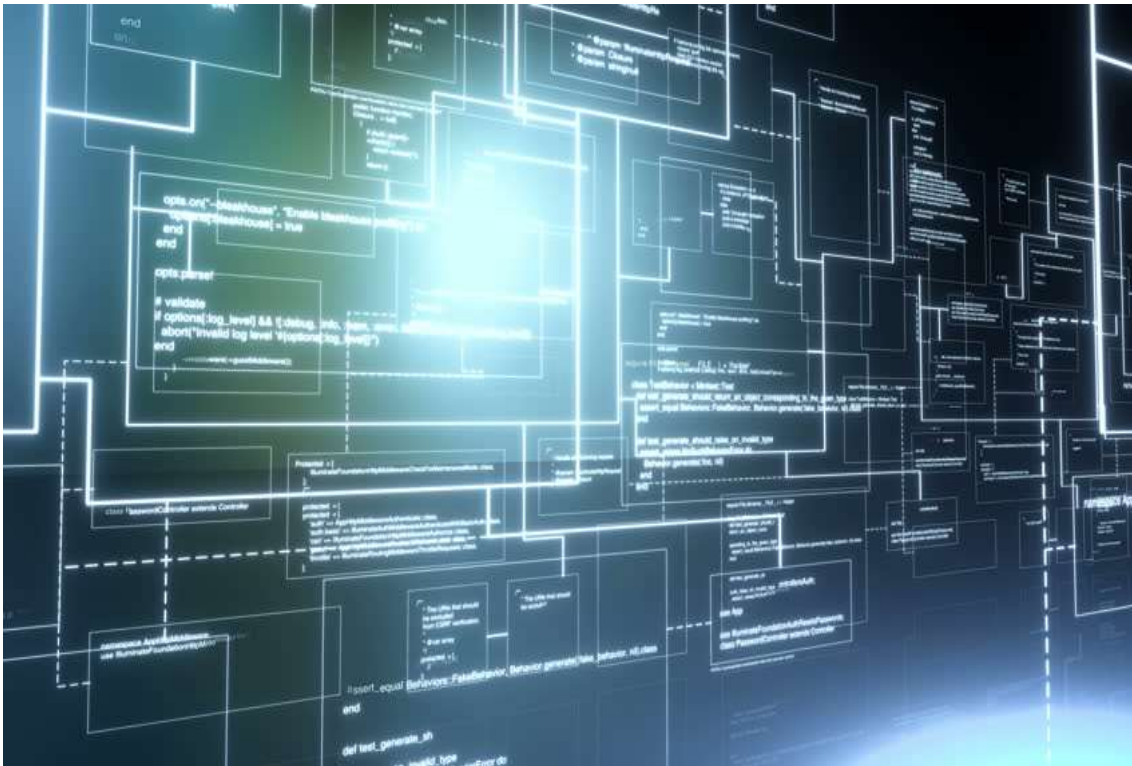
UN SISTEMA DE ARCHIVOS DISTRIBUIDO ES UN TIPO DE SISTEMA DE ALMACENAMIENTO DE DATOS EN EL QUE LOS ARCHIVOS Y LOS RECURSOS DE ALMACENAMIENTO SE DISTRIBUYEN EN MÚLTIPLES NODOS O SERVIDORES INTERCONECTADOS EN UNA RED.

ESTOS SISTEMAS PERMITEN QUE VARIOS USUARIOS O APLICACIONES ACCEDAN Y COMPARTAN ARCHIVOS A TRAVÉS DE LA RED DE MANERA TRANSPARENTE, COMO SI TODOS LOS DATOS ESTUVIERAN EN UNA UBICACIÓN FÍSICA CENTRALIZADA.

SOLO ESTÁN CENTRALIZADOS EN APARIENCIA.



CARACTERÍSTICAS SISTEMAS DE ARCHIVOS DISTRIBUIDOS



LOS DATOS SE ALMACENAN Y DISTRIBUYEN ENTRE VARIOS NODOS EN LA RED PERMITIENDO UN MEJOR RENDIMIENTO Y ESCALABILIDAD, YA QUE EL ACCESO A LOS DATOS SE PUEDE DISTRIBUIR ENTRE VARIOS SERVIDORES.

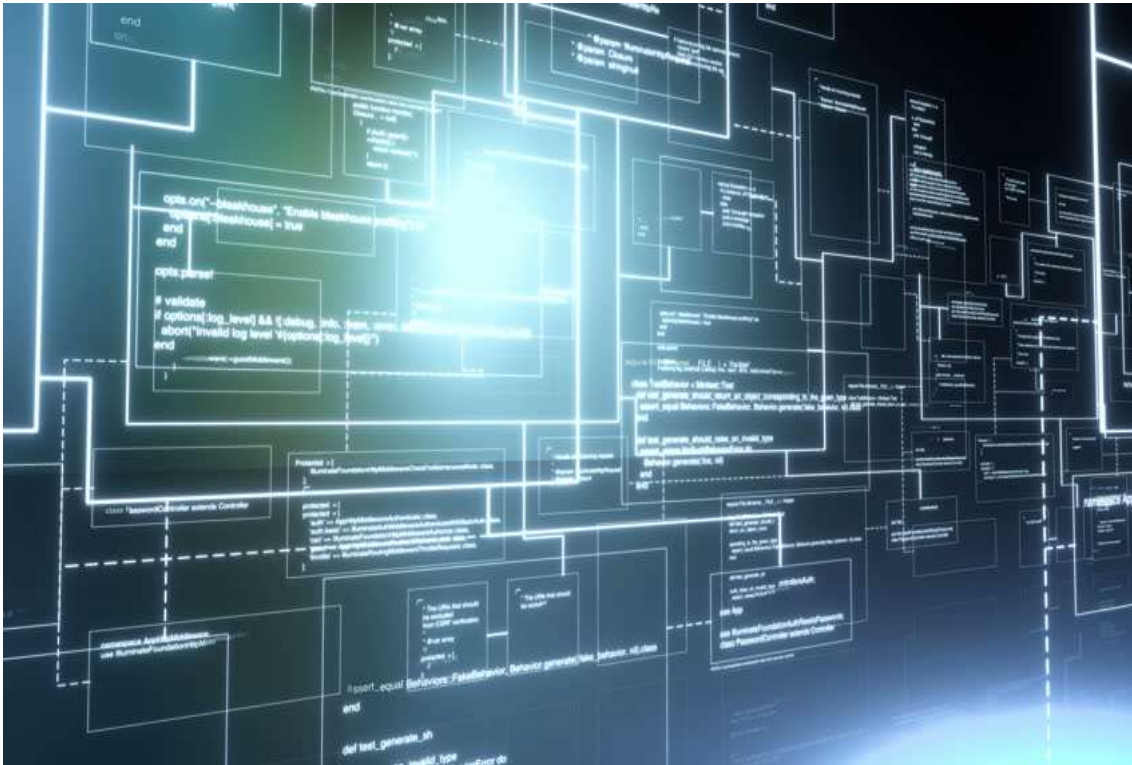
LOS DATOS SE ALMACENAN EN MÚLTIPLES UBICACIONES, PERO LOS USUARIOS Y LAS APLICACIONES PUEDEN ACCEDER A ELLOS COMO SI ESTUVIERAN EN UN ÚNICO SISTEMA DE ARCHIVOS CENTRALIZADO.

LA COMPLEJIDAD DE LA DISTRIBUCIÓN ES OCULTADA A LOS USUARIOS FINALES, POR ESO HABLAMOS DE APARIENCIA CENTRALIZADA.

CUANDO NOS REFERIMOS A UNA RUTA DENTRO DEL SISTEMA DE ARCHIVOS DISTRIBUIDOS, NOS MANEJAMOS IGUAL QUE CON UNA CARPETA DE UN SERVIDOR O MÁQUINA LOCAL. EL CONTENIDO DE ESA RUTA ESTÁ EN REALIDAD DISTRIBUIDO EN LA RED DE SERVIDORES.



CARACTERÍSTICAS SISTEMAS DE ARCHIVOS DISTRIBUIDOS

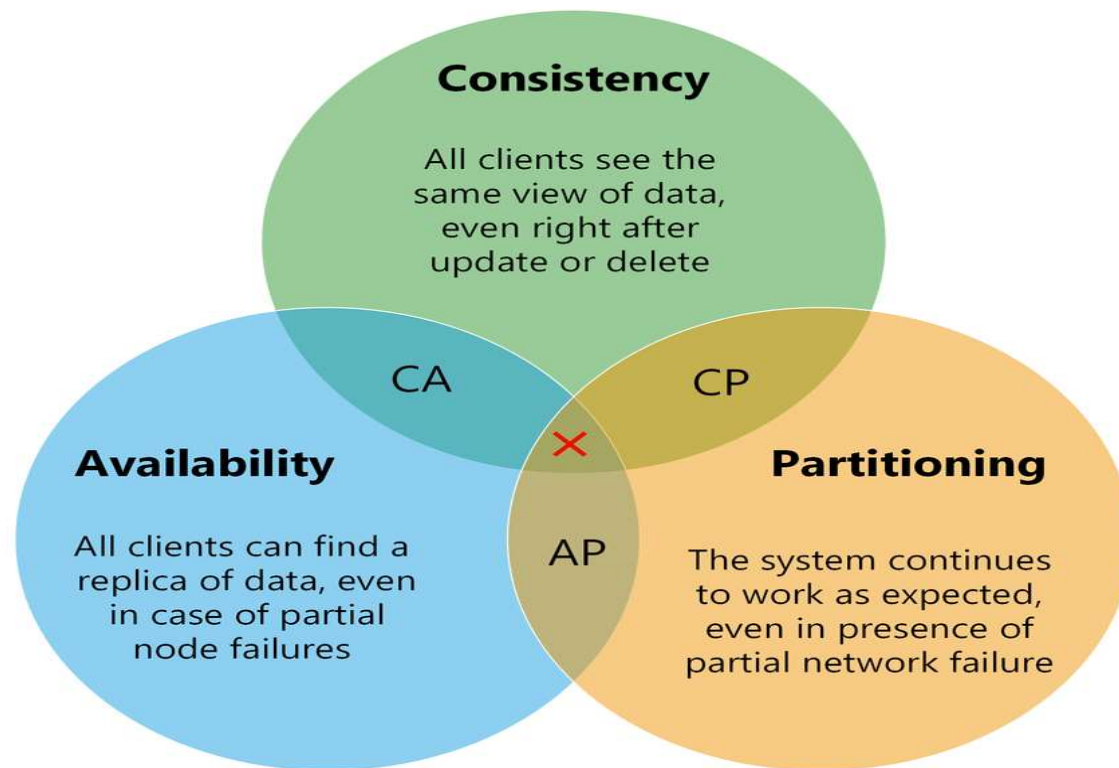


EL SISTEMA DE ARCHIVOS DISTRIBUIDOS TIENE QUE PODER SER MONITOREADO, DETECTAR FALLAS, RECUPERARSE AUTOMÁTICAMENTE, Y SER EN SU CONJUNTO TOLERANTE A LAS FALLAS.

TAMBIÉN DEBEN SER EFICIENTES A LA HORA DE LEER UN ARCHIVO GRANDE DE FORMA COMPLETA Y APPENDEAR NUEVOS DATOS A LOS ARCHIVOS, EN PARTICULAR CUANDO TRABAJAMOS DATOS DE LOGS O SENSORES.

LOS SISTEMAS DE ARCHIVOS DISTRIBUIDOS ADMITEN TANTO EL PARALELISMO COMO EL PROCESAMIENTO POR LOTES DE FORMA NATIVA.

TEOREMA CAP



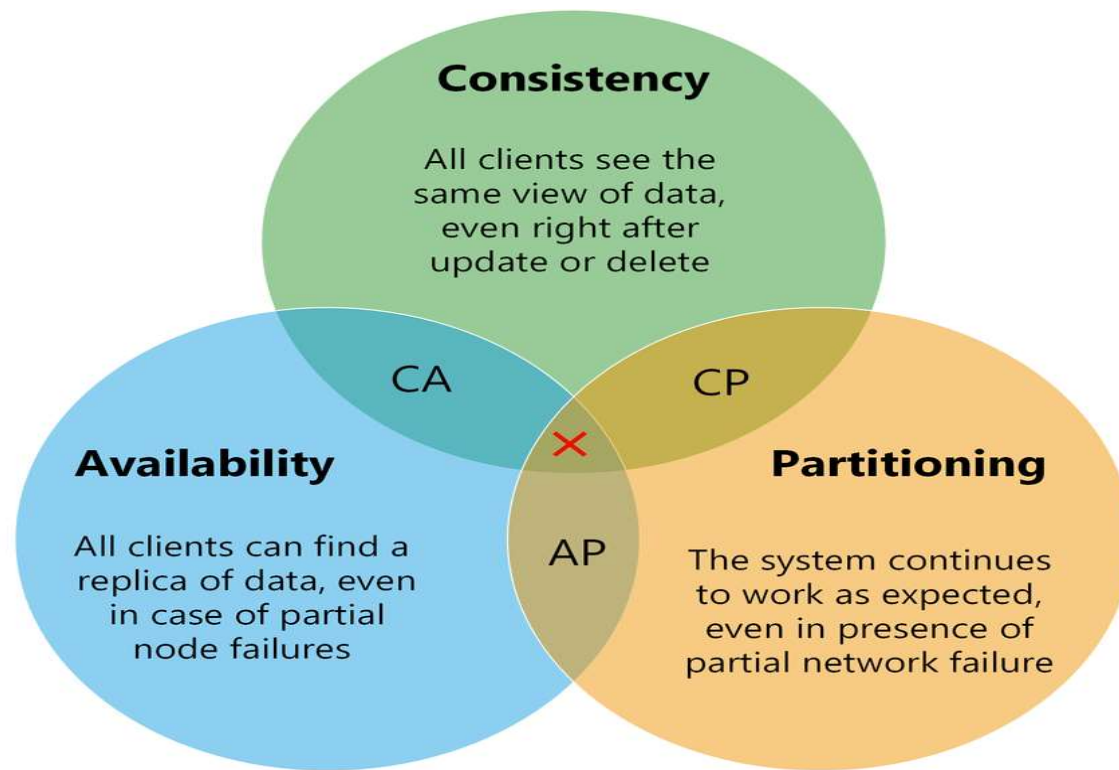
AL PASAR DE SISTEMAS MONOLÍTICOS A SISTEMAS DISTRIBUIDOS, EN OCASIONES DEBEMOS RENUNCIAR PARCIAL O TOTALMENTE A RESTRICCIONES QUE NOS PROVEEN SISTEMAS TRADICIONALES.

LA INTEGRIDAD RELACIONAL QUE PROVEE UN SISTEMA DE BASE DE DATOS RELACIONALES PUEDE NO MANTENERSE. PERDEMOS CLAVES PRIMARIAS Y FORANEAS.

LA INTEGRIDAD DE DOMINIO TAMBIÉN PUEDE PERDERSE. NO TENEMOS PROCEDIMIENTOS QUE GARANTIZEN LA INTEGRIDAD DE LOS DATOS COMO EN SUS SISTEMAS DE ORIGEN.

SE PUEDE PERDER LA INTEGRIDAD ATÓMICA (CONOCIDA COMO PRIMERA FORMA NORMAL) ASÍ COMO PUEDEN PERDERSE LA FORMA NORMAL DE CUALQUIER CONJUNTO DE DATOS.

TEOREMA CAP



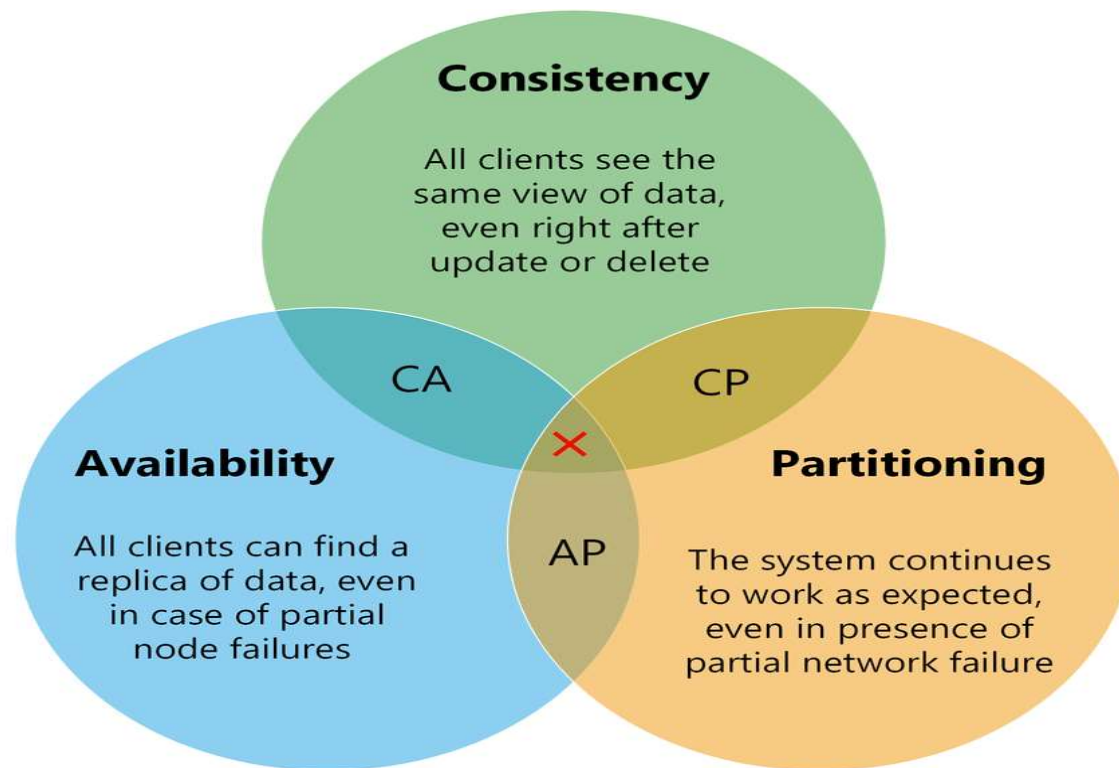
LOS DATOS PUEDEN PASAN A ESTAR EN FORMATO ANIDADO, COMO JSON, CLAVE VALOR, O PASAN A SER DATOS HETEROGÉNEOS.

LOS DATOS HETEROGÉNEOS NO CUMPLEN NECESARIAMENTE CON LA INTEGRIDAD RELACIONAL O DE DOMINIO Y PUEDEN INCLUSIVE NO TENER ESQUEMA. SE PUEDE DECIR QUE SON DATOS DESNORMALIZADOS.

TAMPOCO SE CUMPLE NECESARIAMENTE CON LAS RESTICCIONES ACID (ATOMICITY, CONSISTENCY, ISOLATION, DURABILITY). QUE SERÍA ATOMICIDAD, CONSISTENCIA, AISLACIÓN Y DURABILIDAD EN ESPAÑOL.

POR LO TANTO, AL UTILIZAR SISTEMAS DISTRIBUIDOS DEBEMOS PENSAR EN TÉRMINOS DEL TEOREMA CAP (CONSISTENCY, AVAILABILITY, PARTITION TOLERANCE) CONSISTENCIA, DISPONIBILIDAD Y TOLERANCIA A LA PARTICIÓN.

TEOREMA CAP



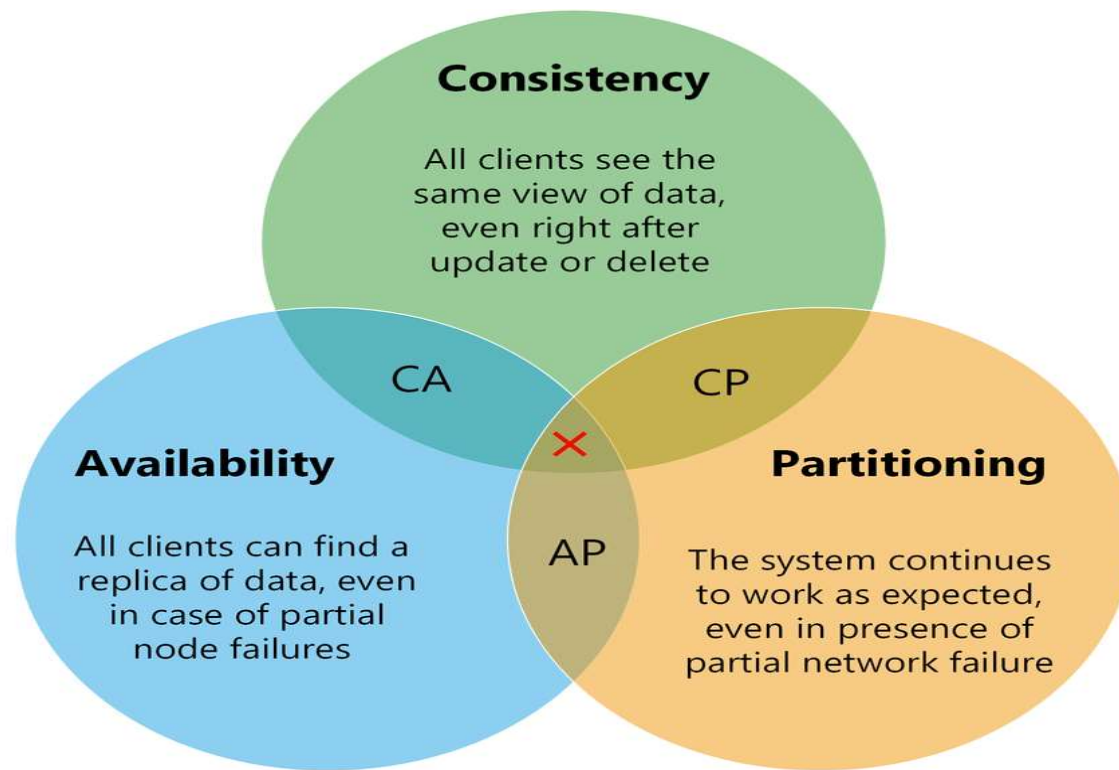
PARA PODER ESCALAR, LOS SISTEMAS DISTRIBUIDOS DEBEN RENUNCIAR A LAS GARANTÍAS EN CUANTO A LA TRANSACCIONALIDAD QUE OFRECEN OTROS SISTEMAS.

LA **CONSISTENCIA ATÓMICA** IMPLICA QUE TODOS EN LA RED VEN LOS MISMOS DATOS AL REQUERIRLOS. INCLUSO DESPUÉS DE ACTUALIZAR O BORRAR DATOS.

LA **DISPONIBILIDAD** IMPLICA QUE TODOS EN LA RED PUEDEN PEDIR EN CUALQUIER MOMENTO LOS DATOS, Y LOS MISMOS SON DE ALTA DISPONIBILIDAD PARA CUALQUIERA, INCLUSO SI ALGÚN NODO FALLA PARCIALMENTE.

LA **TOLERANCIA AL PARTICIONAMIENTO** SE DA CUANDO EL SISTEMA SIGUE FUNCIONANDO, INCLUSO CUANDO FALLA PARCIALMENTE LA RED.

TEOREMA CAP



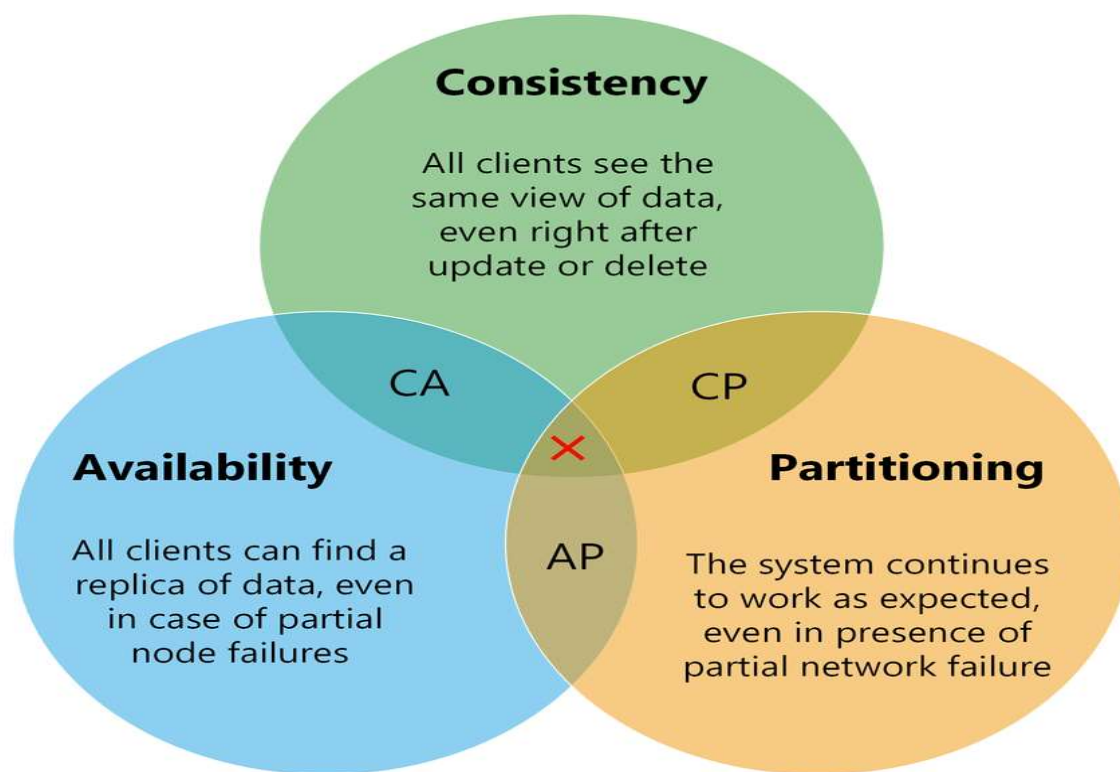
AL ACTUALIZAR LOS DATOS EN UN SISTEMA DISTRIBUIDO, DESDE ALGUNO DE SUS SERVIDORES, ESTA ACTUALIZACIÓN DEBE PROPAGARSE POR EL RESTO DE LA RED, USUALMENTE RESPETANDO UN FACTOR DE REPLICACIÓN PREESTABLECIDO, HASTA TENER LA CANTIDAD DE RÉPLICAS SUFICIENTES.

SI UNA FALLA PARCIAL EN LA RED NO ME PERMITE REPLICAR MIS DATOS, TENGO QUE DETENER EL SISTEMA, HASTA QUE LA RED SE RECUPERE Y PUEDA REPLICAR TODOS LOS DATOS, CASO EN QUE TENGO CONSISTENCIA ATÓMICA, TOLERANCIA AL PARTICIONAMIENTO, PERO NO TENGO ALTA DISPONIBILIDAD.

OTRA OPCIÓN SERÍA QUE LOS SERVIDORES SIGAN RESPONDIENDO, PERO AL ESTAR AISLADA PARTE DE LA RED, LAS RESPUESTAS QUE PUEDO DAR, PODRÍAN SER DISTINTAS. EN ESTE CASO TENDRÍA ALTA DISPONIBILIDAD, TOLERANCIA AL PARTICIONAMIENTO, PERO NO CONSISTENCIA ATÓMICA.



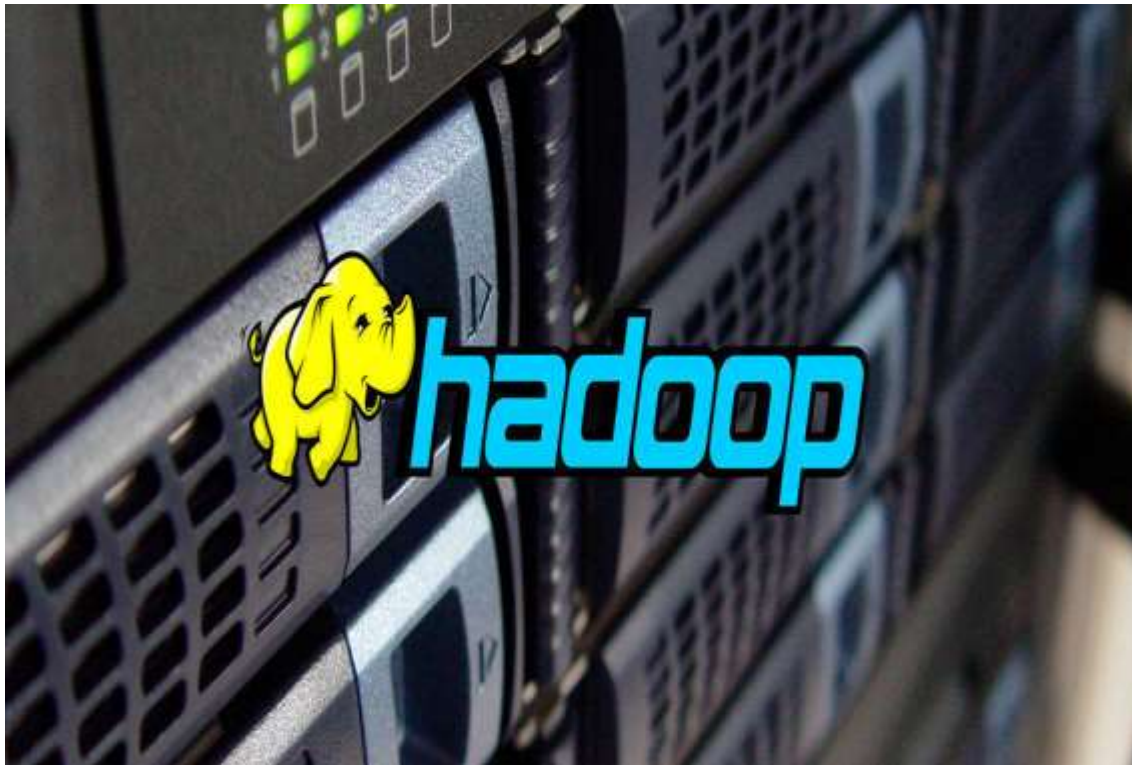
TEOREMA CAP



PARA LA SEGUNDA OPCIÓN LO QUE TENGO ES LLAMADO CONSISTENCIA EVENTUAL. SI DEJO DE RECIBIR DATOS O ACTUALIZACIONES DE LOS MISMOS, EN ALGÚN MOMENTO, EVENTUALMENTE, TENDRÉ CONSISTENCIA ATÓMICA, PORQUE LOS DATOS YA SE HABRÁN PROPAGADO POR TODA LA RED.

SI DESEO QUE MI SISTEMA TENGA CONSISTENCIA ATÓMICA Y ALTA DISPONIBILIDAD, NO PUEDO TENER TOLERANCIA AL PARTICIONADO, PORQUE AL QUEDAR DESCONECTADA PARTE DE LA RED DEBERÍA TOMAR UNA DE LAS OPCIONES ANTERIORES.

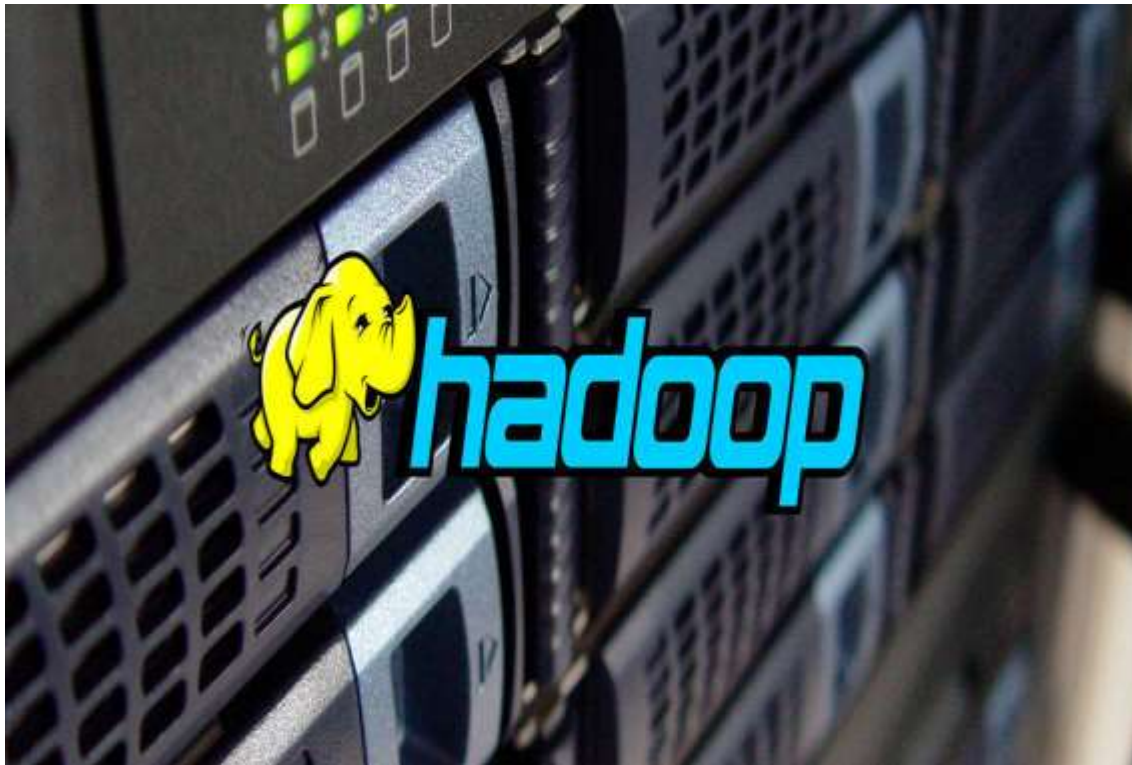
¿QUÉ ES HADOOP?



HADOOP ES UN FRAMEWORK DE CÓDIGO ABIERTO PARA ALMACENAR DATOS Y EJECUTAR APLICACIONES EN CLÚSTERES DE HARDWARE COMERCIAL.

PROPORCIONA ALMACENAMIENTO MASIVO PARA CUALQUIER TIPO DE DATOS, TIENE UN ENORME PODER DE PROCESAMIENTO Y LA CAPACIDAD DE PROCESAR TAREAS O TRABAJOS CONCURRENTES VIRTUALMENTE ILIMITADOS.

¿QUÉ ES HADOOP?



DOUG CUTTING, UNO DE LOS CREADORES DE HADOOP, ELIGE EL NOMBRE POR UN ELEFANTE DE JUGUETE QUE PERTENECÍA A SU HIJO, Y HABÍA SIDO BAUTIZADO CON ESTE NOMBRE.

LA ROBUSTEZ QUE SIMBOLIZA EL ELEFANTE TAMBIÉN COLABORÓ PARA ELEGIR ESTE ANIMAL COMO LOGO PARA ESTA TECNOLOGÍA, YA QUE ESA ES LA IMAGEN QUE SE QUIERE TRANSMITIR COMO PRINCIPAL CUALIDAD.

HISTORIA DE HADOOP



Mike cafarella and Doug Cutting



EN 2003 DOUG CUTTING, CREADOR DE LUCENE, JUNTO A MIKE CAFARELLA, ESTABAN TRABAJANDO EN NOUCH, UN MOTOR DE BÚSQUEDA WEB OPEN SOURCE.

POR EL TAMAÑO DE LO QUE ERA LA WEB YA EN 2003 SE NECESITABAN DESARROLLAR UN PRODUCTO QUE FUERA ESCALABLE.

**¿SE ENFRENTABAN
A UN PROBLEMA
DE BIG DATA?**



SI

HISTORIA DE HADOOP



Mike Cafarella and Doug Cutting



¿SE ENFRENTABAN A UN PROBLEMA DE BIG DATA?

SI, NO SE PUEDE RECORRER TODA LA WEB, INDEXARLA Y GUARDAR TODA LA METADATA DE CADA SITIO CON MÉTODOS CONVENCIONALES.

HISTORIA DE HADOOP



Mike Cafarella and Doug Cutting



¿CUÁL FUE LA SOLUCIÓN?

A PARTIR DE CIERTOS PAPERS PUBLICADOS POR GOOGLE DE COMO HACÍAN PARA TRABAJAR INTERNAMENTE, DOUG Y MIKE DECIDIERON REPLICAR ESE TRABAJO PARA HACERLO OPEN SOURCE Y DEJARLO DISPONIBLE PARA TODA LA COMUNIDAD.

EN PARTICULAR EL SISTEMA DE ARCHIVOS TFS Y EL FRAMEWORK MAP REDUCE.

PARA 2006 YAHOO INVITÓ EN EL PROYECTO, PASANDO CUTTING A FORMAR PARTE DE LA EMPRESA.



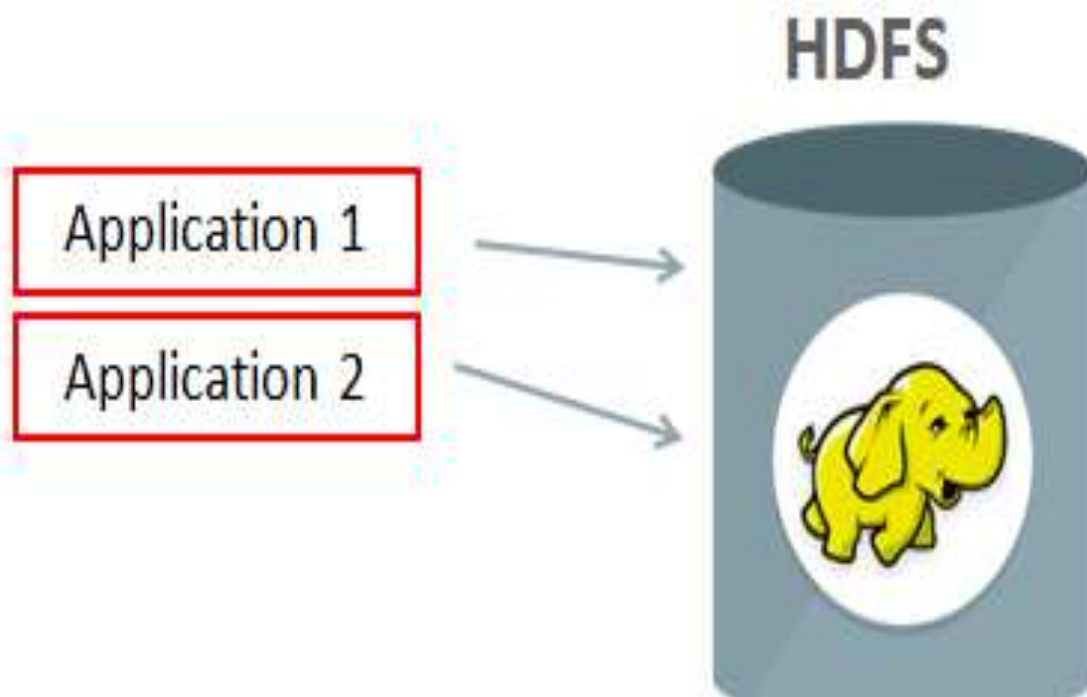
HDFS (HADOOP DISTRIBUTED FILE SYSTEM)

ES UN SISTEMA DE ARCHIVOS PORTÁTIL, ESCALABLE Y DISTRIBUIDO BASADO EN JAVA, DISEÑADO PARA ABARCAR GRANDES CLÚSTERES DE SERVIDORES.

SU DISEÑO SE BASA EN TFS, EL SISTEMA DE ARCHIVOS DE GOOGLE.

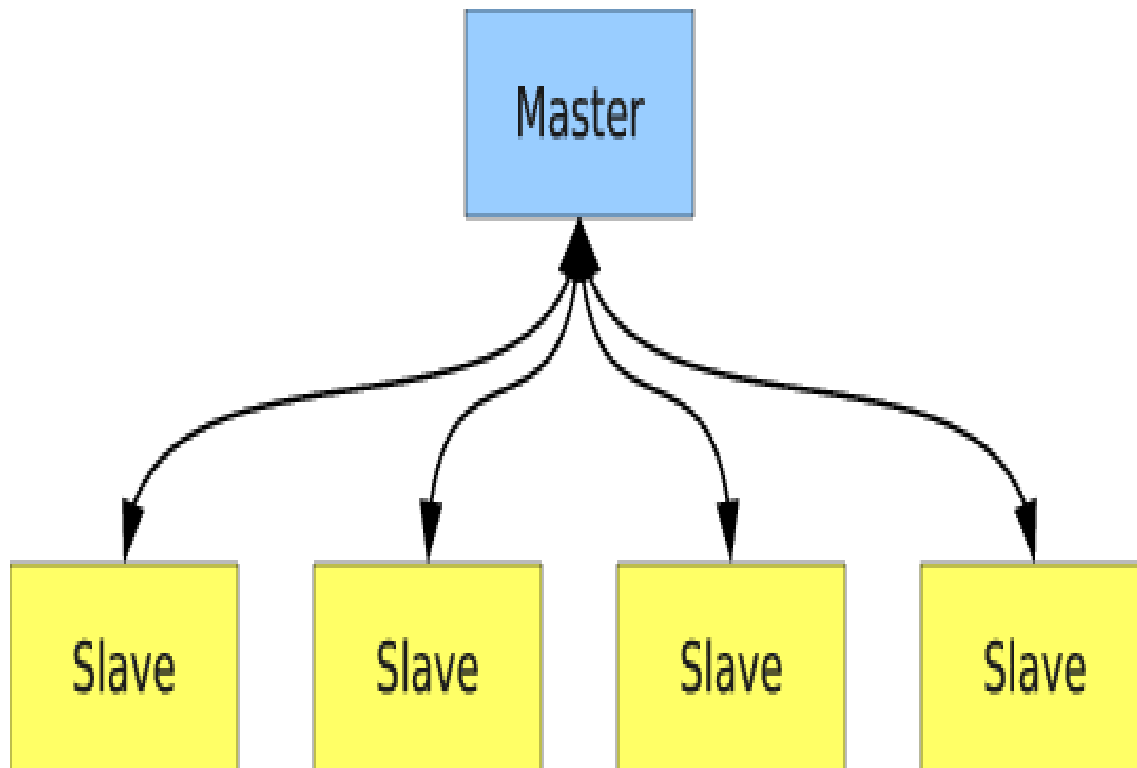
HDFS PUEDE ALMACENAR UNA GRAN CANTIDAD DE DATOS Y PROPORCIONAR ACCESO A VARIOS CLIENTES DISTRIBUIDOS A TRAVÉS DE UNA RED.

HDFS SOBRESALE EN SU CAPACIDAD PARA ALMACENAR ARCHIVOS GRANDES DE MANERA CONFIABLE Y ESCALABLE.





HDFS (MODELO MASTER SLAVE)



HDFS UTILIZA EL ESQUEMA CONOCIDO COMO MASTER SLAVE PARA COORDINAR LOS NODOS DEL SISTEMA.

A DIFERENCIA DE UNA ARQUITECTURA DESCENTRALIZADA COMO P2P, SE CONSIDERA ESTA ARQUITECTURA COMO UNA CENTRALIZADA, DONDE UN NODO ES MÁS IMPORTANTE QUE LOS DEMÁS.

ES UN MODELO DE COMUNICACIÓN O CONTROL ASIMÉTRICO DONDE UN DISPOSITIVO O PROCESO CONTROLA UNO O MÁS DISPOSITIVOS O PROCESOS Y SIRVE COMO SU CENTRO DE COMUNICACIÓN.

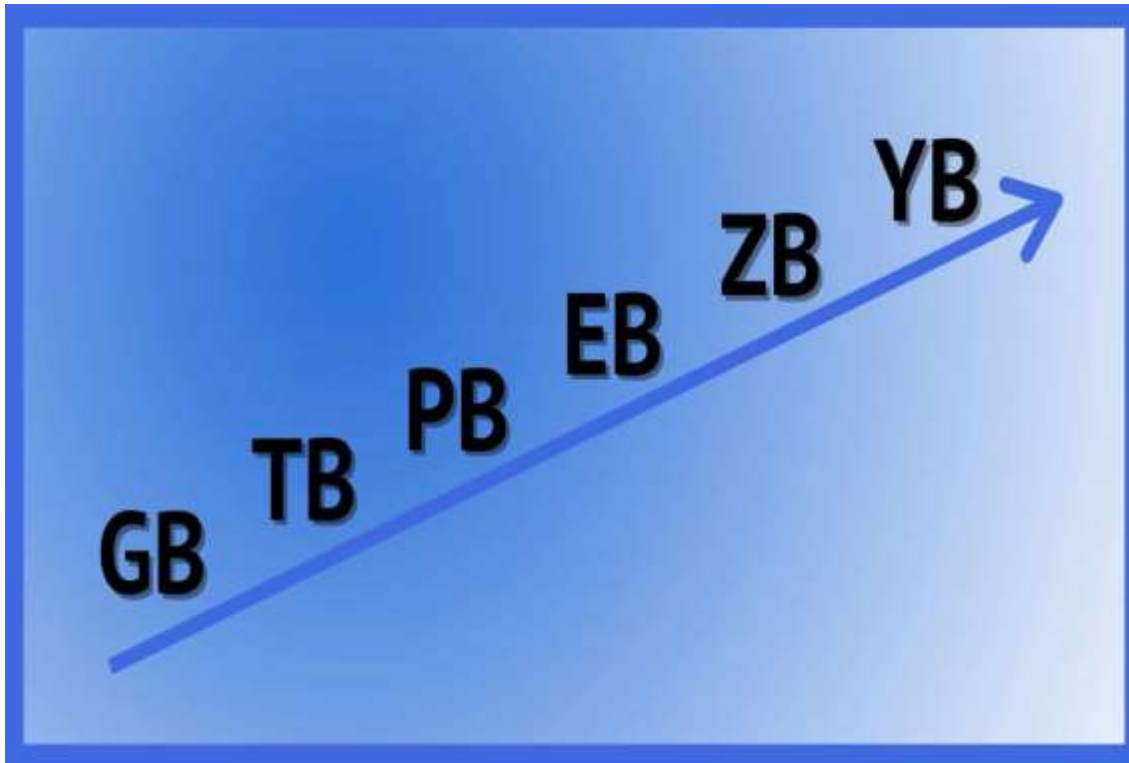
EN ALGUNOS SISTEMAS, SE SELECCIONA UN MAESTRO DE UN GRUPO DE DISPOSITIVOS ELEGIBLES, Y LOS OTROS DISPOSITIVOS ACTÚAN EN EL PAPEL DE ESCLAVOS SIGUIENDO SUS ÓRDENES.



Si falla el nodo maestro, un nodo esclavo sustituye su lugar.

HDFS (MODELO MASTER SLAVE)

HDFS (ALMACENAMIENTO)



HDFS ESTÁ DISEÑADO PARA ALMACENAR MUCHA INFORMACIÓN.

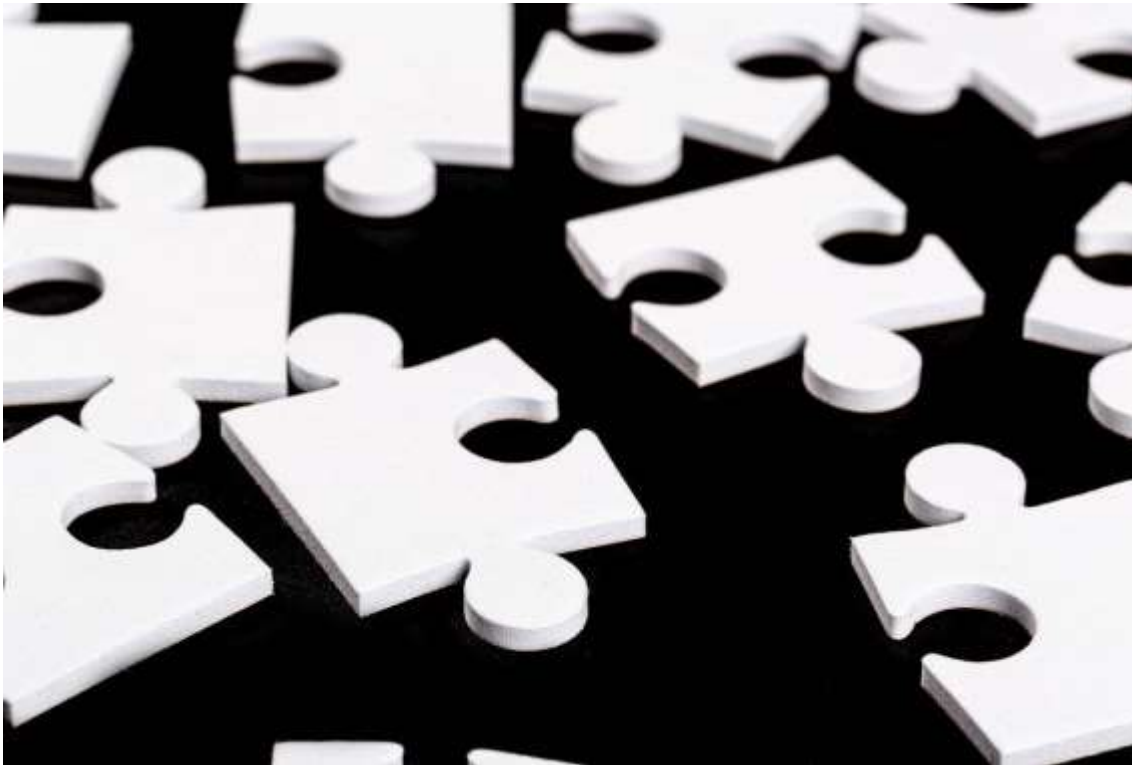
GENERALMENTE PETABYTES, GIGABYTES Y TERABYTES.

ESTO SE LOGRA MEDIANTE EL USO DE UN SISTEMA DE ARCHIVOS ESTRUCTURADO EN BLOQUES.

LOS ARCHIVOS INDIVIDUALES SE DIVIDEN EN BLOQUES DE TAMAÑO FIJO QUE SE ALMACENAN EN MÁQUINAS EN TODO EL CLÚSTER. NORMALMENTE ESTOS BLOQUES PESAN 64 O 128 MB.

LOS ARCHIVOS COMPUESTOS DE VARIOS BLOQUES GENERALMENTE NO TIENEN TODOS SUS BLOQUES ALMACENADOS EN UNA SOLA MÁQUINA.

HDFS (REPLICACIÓN)



HDFS GARANTIZA LA INTEGRIDAD DE LOS DATOS MEDIANTE LA REPLICACIÓN DE BLOQUES Y LA DISTRIBUCIÓN DE LAS RÉPLICAS EN TODO EL CLÚSTER.

EL FACTOR DE REPLICACIÓN PREDETERMINADO ES TRES, LO QUE SIGNIFICA QUE CADA BLOQUE EXISTE TRES VECES EN EL CLÚSTER.

LA REPLICACIÓN PERMITE LA DISPONIBILIDAD DE DATOS INCLUSO CUANDO FALLAN LAS MÁQUINAS.

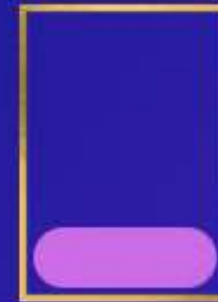
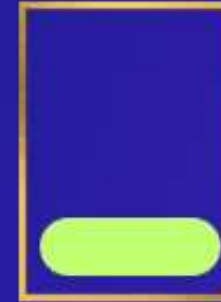
**Tengo un
archivo**



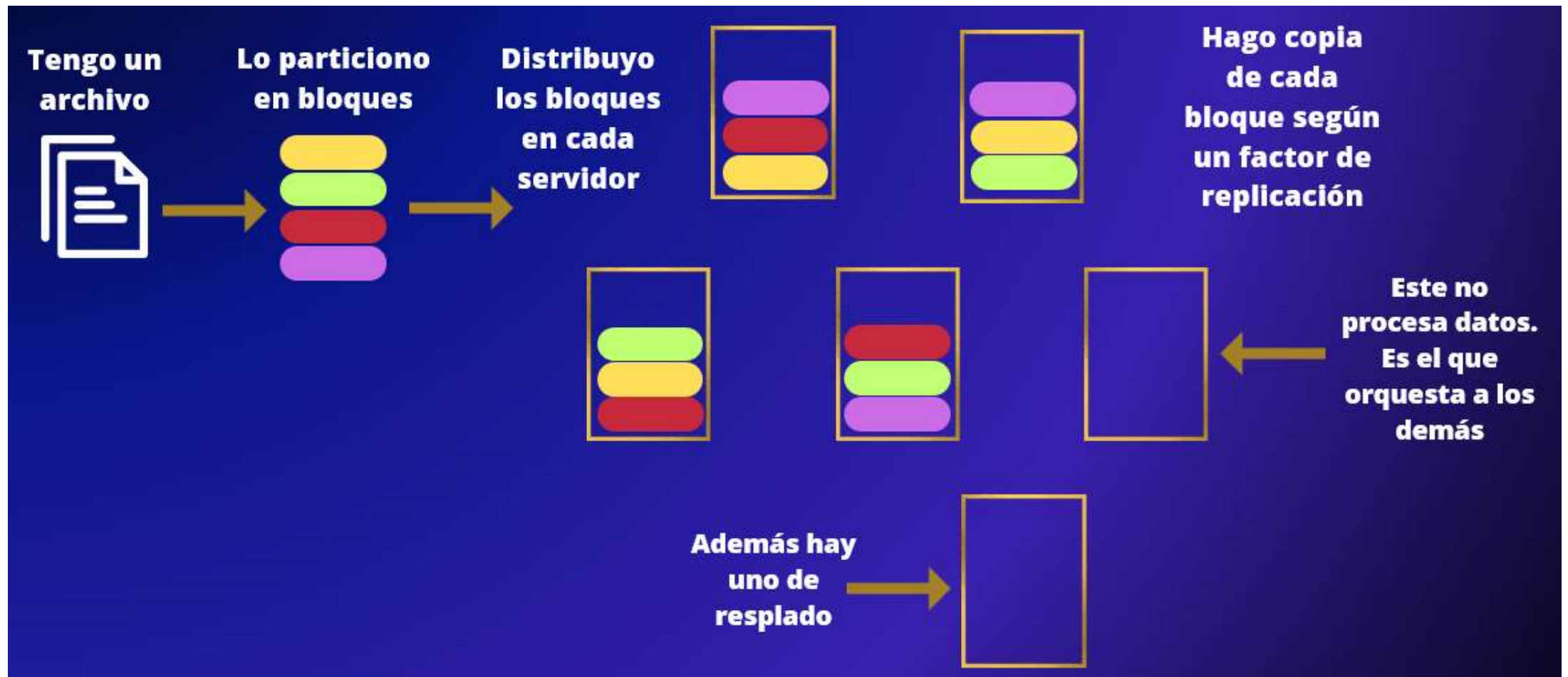
**Lo particiono
en bloques**



**Distribuyo
los bloques
en cada
servidor**



PARTICIÓN EN BLOQUES Y REPLICACIÓN

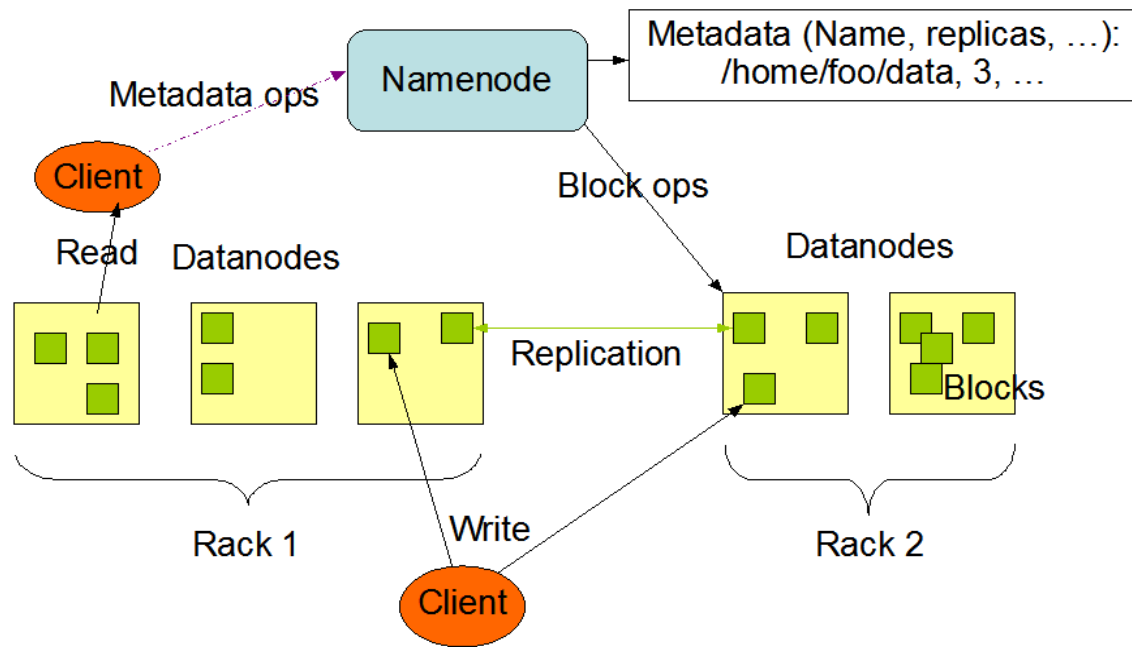


PARTICIÓN EN BLOQUES Y REPLICACIÓN



HDFS (ARQUITECTURA)

HDFS Architecture



EL DISEÑO ARQUITECTÓNICO DE HDFS SE COMPONE DE DOS PARTES:

EL NAMENODE QUE CONTIENE LOS METADATOS PARA EL SISTEMA DE ARCHIVOS, Y UNO O MÁS DATANODES QUE ALMACENAN LOS BLOQUES QUE COMPONEN LOS ARCHIVOS.

ESTOS SON LOS NOMBRES QUE SE UTILIZAN EN HADOOP PARA EL NODO MASTER Y LOS NODOS SLAVE.

LOS NAMENODE Y DATANODE PUEDEN EJECUTARSE EN UNA SOLA MÁQUINA, PERO LOS CLÚSTERES DE HDFS COMÚNMENTE CONSTAN DE UN SERVIDOR DEDICADO QUE EJECUTA EL NAMENODE Y VARIAS MÁQUINAS QUE EJECUTAN LOS DATANODES.



HDFS (NAMENODE)

NameNode



DataNode



NAMENODE ES LA MÁQUINA MÁS IMPORTANTE EN HDFS.

ALMACENA METADATOS PARA TODO EL SISTEMA DE ARCHIVOS:
NOMBRES DE ARCHIVOS, PERMISOS Y LA UBICACIÓN DE CADA
BLOQUE DE CADA ARCHIVO.

PARA PERMITIR UN ACCESO RÁPIDO A ESTA INFORMACIÓN, EL
NAMENODE ALMACENA TODA LA ESTRUCTURA DE METADATOS
EN MEMORIA Y NO EN DISCO.

HDFS (NAMENODE)

NameNode



DataNode



NAMENODE TAMBIÉN REALIZA UN SEGUIMIENTO DEL FACTOR DE REPLICACIÓN DE LOS BLOQUES, LO QUE GARANTIZA QUE LAS FALLAS DE LA MÁQUINA NO PROVOQUEN LA PÉRDIDA DE DATOS.

SE ASEGURA QUE LOS BLOQUES ESTÉN REPLICADOS LA CANTIDAD DE VECES QUE FUE INDICADO.

REPLICA LOS BLOQUES DE FORMA ASINCRÓNICA EN CASO NO ENCUENTRE LA CANTIDAD DE BLOQUES SUFICIENTES.

HDFS (NAMENODE)



NameNode

DataNode



¿QUÉ PASA SI EL NAMENODE FALLA?

SE PUEDE USAR UN NAMENODE SECUNDARIO PARA GENERAR INSTANTÁNEAS (SNAPSHOTS DE RESPALDO) DE LAS ESTRUCTURAS DE MEMORIA DEL NAMENODE PRINCIPAL, LO QUE REDUCE EL RIESGO DE PÉRDIDA DE DATOS SI EL NAMENODE DEJA DE FUNCIONAR.

VAMOS A TENER ENTONCES DOS MODALIDADES DE NAMENODE:

- ACTIVE NAMENODE
- STANDBY NAMENODE

MÁS ADELANTE, HDFS INCORPORÓ **NAMENODES FEDERADOS**. SON NAMENODES QUE CORREN AL MISMO TIEMPO Y SE REPARTEN LA CARGA DE TRABAJO SIN SOLAPAR SUS TAREAS.

HDFS (DATANODE)



NameNode



DataNode



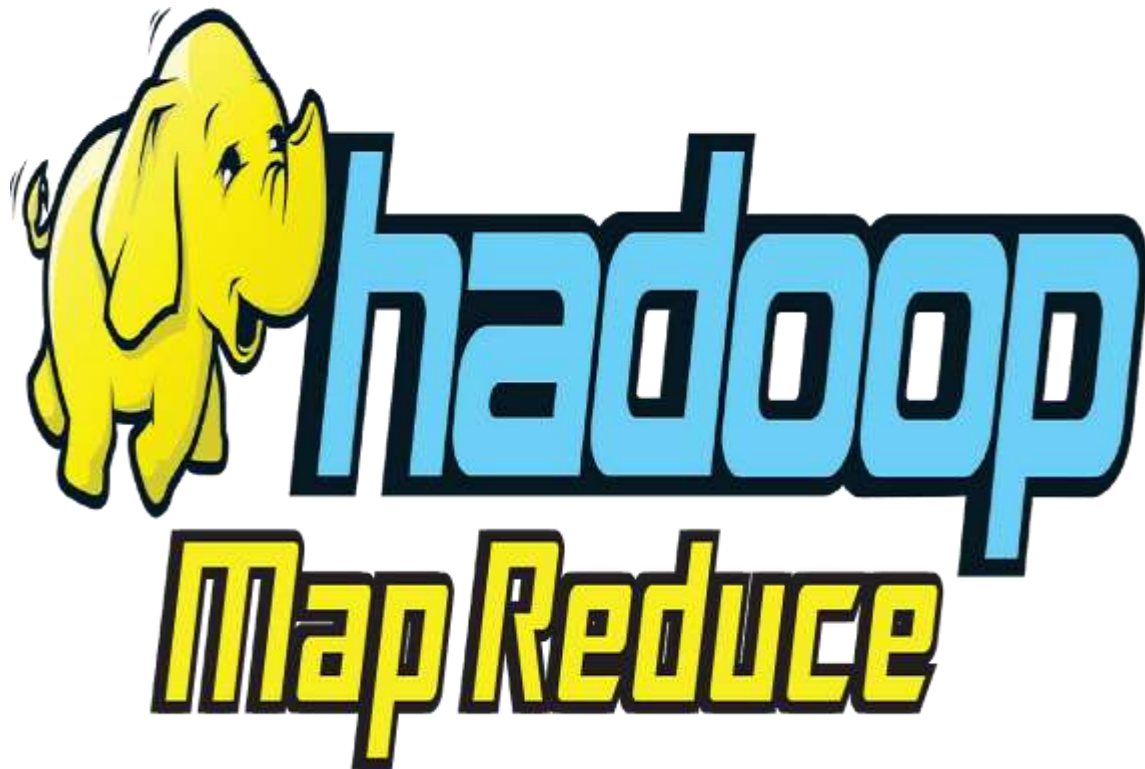
LAS MÁQUINAS QUE ALMACENAN LOS BLOQUES DENTRO DE HDFS SE DENOMINAN DATANODES.

LOS DATANODES SUELEN SER SERVIDORES CON GRANDE CAPACIDAD DE ALMACENAMIENTO.

A DIFERENCIA DE NAMENODE, HDFS CONTINUARÁ FUNCIONANDO NORMALMENTE SI FALLA UN DATANODE. CUANDO UN DATANODE FALLA, EL NAMENODE REPLICARÁ LOS BLOQUES PERDIDOS PARA GARANTIZAR QUE CADA BLOQUE CUMPLA CON EL FACTOR DE REPLICACIÓN MÍNIMO.

SOLO AL CREAR UN NUEVO ARCHIVO EN HDFS LOS DATANODES SE COMUNICAN ENTRE SI AL CREAR UN PIPELINE DE REPLICACIÓN. EI PRIMER DATA NODE QUE RECIBE EL BLOQUE LO PORPAGA A SUS PARES.

HDFS (MAPREDUCE)



¿CÓMO FUNCIONA MAP REDUCE?

EN VEZ QUE LOS DATOS VENGAN AL CÓDIGO. EL CÓDIGO VA A LOS DATOS.



HDFS (MAPREDUCE)

A large, stylized graphic of the Hadoop logo, including a yellow elephant and the word "hadoop" in blue and yellow, serving as a background for the table.

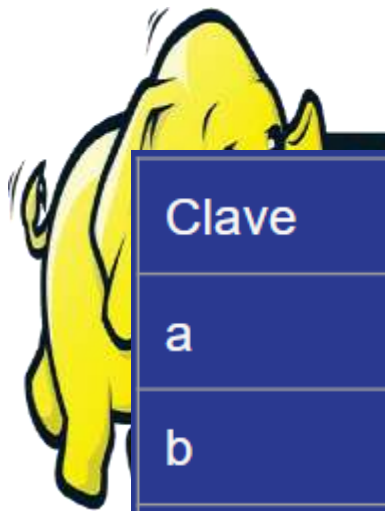
Clave	Valor
a	2
a	3
b	4
c	2
b	1

EJEMPLO.

TENGO EL SIGUIENTE DATA SET EN LA IMAGEN AL COSTADO
QUIERO SABER LA SUMA DE LOS VALORES PARA CADA CLAVE.



HDFS (MAPREDUCE)



Clave	Valor
a	2, 3
b	4, 1
c	2



EJEMPLO.

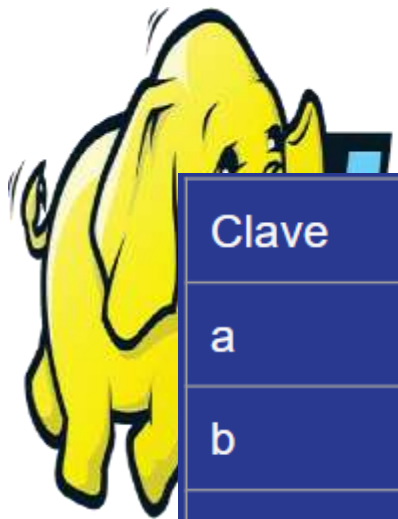
TENGO EL SIGUIENTE DATA SET EN LA IMAGEN AL COSTADO

QUIERO SABER LA SUMA DE LOS VALORES PARA CADA CLAVE

MAP

MAPEO CADA CLAVE Y VOY RECOLECTANDO LOS VALORES QUE CORRESPONDEN A CADA CLAVE.

HDFS (MAPREDUCE)



Clave	Valor
a	5
b	5
c	2

EJEMPLO.

TENGO EL SIGUIENTE DATA SET EN LA IMAGEN AL COSTADO

QUIERO SABER LA SUMA DE LOS VALORES PARA CADA CLAVE

MAP

MAPEO CADA CLAVE Y VOY RECOLECTANDO LOS VALORES QUE CORRESPONDEN A CADA CLAVE.

REDUCE

LUEGO REDUZCO LA INFORMACIÓN. EN ESTE CASO MEDIANTE LA SUMA DE LOS VALORES.

HDFS (MAPREDUCE)



OTRO EJEMPLO ES LA FUNCIÓN WORD COUNT QUE ME INDICA LAS PALABRAS QUE HAY EN UNO O VARIOS DOCUMENTOS, Y LA CANTIDAD DE OCURRENCIAS DE CADA UNA.

ESTE ES EL MISMO PRINCIPIO QUE UTILIZA HADOOP PARA CONSULTAR LA INFORMACIÓN DISTRIBUIDA A LO LARGO DE VARIOS SERVIDORES.

HDFS (MAPREDUCE)



MAP REDUCE ES UN ALGORITMO QUE PERMITE PROCESAR GRANDES VOLÚMENES DE DATOS DIVIDIENDO EL TRABAJO (JOB) EN PEQUEÑAS TAREAS INDEPENDIENTES (TASKS) Y EJECUTANDO (CON EXECUTORS) ESAS TAREAS EN PARALELO A TRAVÉS DE UN CLÚSTER.

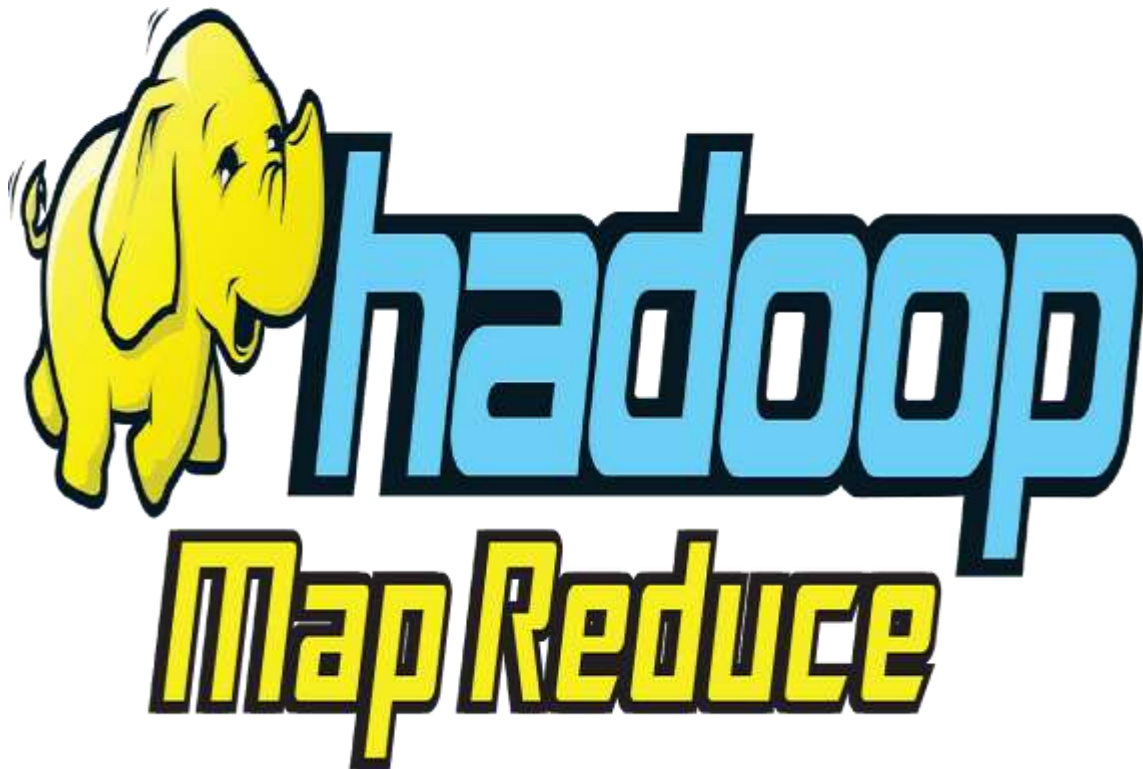
LOS EXECUTORS ESTÁN EN LOS DATANODES.

EL NAMENODE ASIGNA LOS TASKS A CADA EXECUTOR.

EL ALGORITMO ESTÁ INSPIRADO EN LA PROGRAMACIÓN FUNCIONAL TOMANDO LAS FUNCIONES DE MAPEO Y REDUCCIÓN, QUE SON COMÚNMENTE UTILIZADAS PARA PROCESAR LISTAS DE DATOS.



HDFS (MAPREDUCE)



EL NAMENODE TAMBIEN CONTIENE UN JOBTRACKER Y CADA DATA NODE CONTIENE UN TASKTRACKER. SE ENCARGAN DE SEGUIR EL CUMPLIMIENTO DEL JOB Y DE CADA TASK RESPECTIVAMENTE.



¿CÓMO FUNCIONA MAPREDUCE?

Función Mapper

MAPEA DISTINTOS PARES DE CLAVE VALOR. PROCESA SECUENCIALMENTE ESTOS PARES DE CLAVEVALOR, PRODUCIENDO NINGÚNO O VARIOS OUTPUTS. DEPENDIENDO SI HAY O NO DATOS PARA LAS CLAVES BUSCADAS.

Shuffle and Sort

EL PROCESO DE MOVER LOS OUTPUTS DEL MAPPER A LOS REDUCTORES SE LLAMA SHUFFLE. SE ENCARGA DE QUE TODOS LOS VALORES DE LA MISMA CLAVE, VAYAN AL MISMO REDUCTOR. POR DEFAULT SE UTILIZA HASHING. LUEGO SE ORDENAN LOS DATOS.

Función Reduce

LA FUNCIÓN REDUCE ITERA POR LOS DATOS PARA CADA CLAVE, APLICANDO FUNCIONES DE AGREGACIÓN Y DEVOLVIENDO LUEGO LOS OUTPUT.

SISTEMAS DE BUCKETS



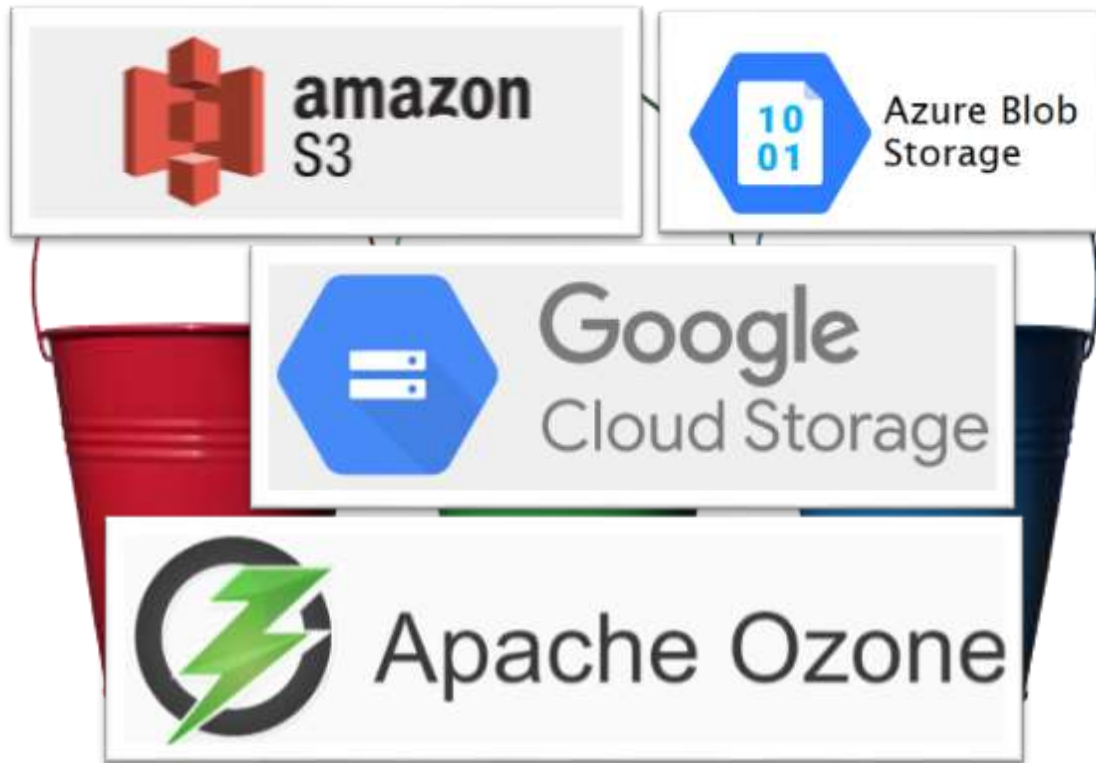
UN BALDE (BUCKET) O ALMACENADOR DE OBJETOS (OBJECT STORAGE), ES UN CONTENEDOR VIRTUAL, NORMALMENTE ALMACENADO EN LA NUBE, AUNQUE TAMBIÉN SE PUEDE TENER ON PREMISE, DONDE LOS USUARIOS ACCEDEN A TRAVÉS DE APIS O URLS ESPECIFICAS.

EN LA NUBE SE OFRECE TÍPICAMENTE COMO UN SERVICIO. POR LO QUE LOS USUARIOS DIRECTAMENTE USAN SIN INSTALARLOS.

CADA BUCKET TIENE UN IDENTIFICADOR ÚNICO DENTRO DEL PROVEEDOR CLOUD.

NO ADMITE DENTRO DE SI MÍSMO UNA JERARQUÍA COMO LOS ARCHIVOS, SINO QUE LAS SIMULA CON UNA URL PROPIA DEL IDENTIFICADOR UTILIZANDO “/”, LA BARRA PARA CADA RUTA DE CADA OBJETO.

SISTEMAS DE BUCKETS



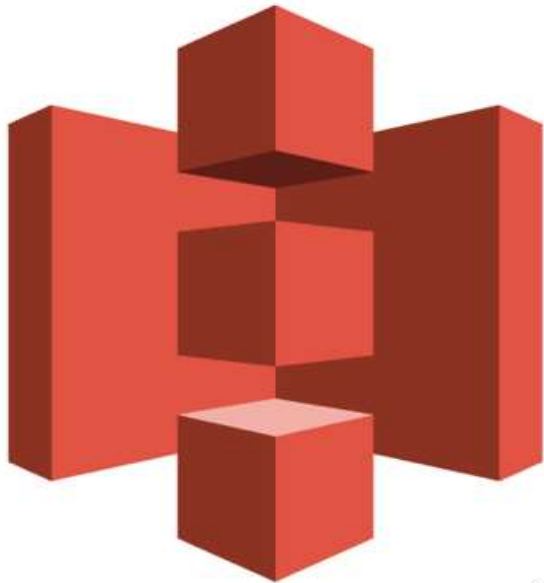
EN EL PARADIGMA DE BUCKETS TRATAMOS LOS ARCHIVOS COMO OBJETOS.

CADA OBJETO TIENE SU PROPIA CLAVE.

LOS BUCKETS SON UTILIZADOS COMO REPOSITORIOS DE GRANDES VOLUMENES DE DATOS EN ORGANIZACIONES, PERO TAMBIÉN PARA CASOS DE USO QUE NO SON DE BIG DATA, COMO SUBIR DATOS DE UNA WEB QUE SE PRECARGAN EN EL BUCKET.

PODEMOS USAR BUCKETS PARA EL ALMACENAMIENTO DE LOS DATOS DE NUESTRO DATALAKE, YA QUE AL SER ALTAMENTE ESCALABLES, SIRVEN PARA EL USO EN PROBLEMAS DE BIG DATA.

SISTEMAS DE BUCKETS (AWS S3)



Amazon S3

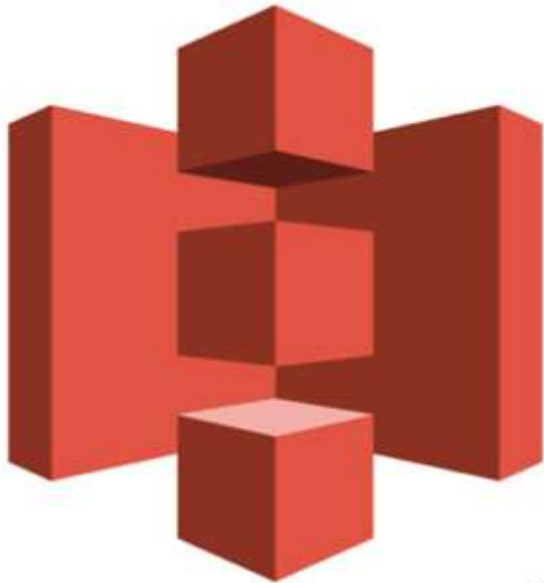
AMAZON WEB SERVICES OFRECE S3 COMO SISTEMA DE ALMACENAMIENTO DE OBJETOS.

LOS OBJETOS SON ORGANIZADOS EN BUCKETS.

CADA BUCKET TIENE UN ID ÚNICO DENTRO DE AWS.

CADA OBJETO DENTRO DEL BUCKET, TAMBIÉN TIENE UN ID ÚNICO, DENTRO DEL BUCKET, POR LO QUE JUNTANDO EL ID DEL BUCKET CON EL DEL OBJETO, CADA OBJETO TIENE UN ID ÚNICO DENTRO DE AWS.

SISTEMAS DE BUCKETS (AWS S3)



Amazon S3

EL OBJETO PUEDE SER CUALQUIER ARCHIVO DE CUALQUIER FORMATO, SEA ESTE ESTRUCTURADO O NO. LA DIFERENCIA CON LOS ARCHIVOS ES QUE LOS MISMOS TIENEN UNA JERARQUÍA DENTRO DE CARPETAS, DE LA CUÁL LOS OBJETOS CARECEN.

UN OBJETO PUEDE PESAR HASTA 5TB, Y PUEDO TENER HASTA 100 BUCKETS EN SIMULTANEO O PEDIR AUTORIZACIÓN A AWS PARA TENER MÁS.

POR MÁS QUE UN OBJETO TENGA UN LÍMITE DE 5TB UN CONJUNTO DE DATOS PUEDE ESTAR EN VARIOS OBJETOS. LA TÉCNICA DE SEPARAR UN CONJUNTO DE DATOS EN PEDAZOS DE 5TB SE LE LLAMA SHARDING.

S3 PROMETE PERDER CADA AÑO, MENOS DE UN OBJETO CADA MIL MILLONES.

SISTEMAS DE BUCKETS (AZURE BLOB STORAGE)



Microsoft Azure
Blob Storage



A DIFERENCIA DE AWS S3, LA ARQUITECTURA DE AZURE BLOB STORAGE ESTÁ DOCUMENTADA EN VARIOS PAPERS CIENTÍFICOS.

LOS IDENTIFICADORES SON DE LA CUENTA, DEL CONTENEDOR (TAMBIÉN LLAMADO PARTICIÓN), Y DEL BLOB.

LOS BLOQUES QUE FORMAN UN OBJETO SON EXPUESTOS AL USUARIO FINAL, AL IGUAL QUE EN HDFS.

SISTEMAS DE BUCKETS (AZURE BLOB STORAGE)



Microsoft Azure
Blob Storage



AZURE OFRECE TRES TIPOS DE BLOBS, OPTIMIZADOS PARA DIFERENTES TAREAS:

LOS **BLOBS DE BLOQUES** ALMACENAN DATOS DE TEXTO Y BINARIOS. ESTÁN COMPUESTOS POR BLOQUES DE DATOS QUE PUEDEN GESTIONARSE DE FORMA INDIVIDUAL.

LOS **APPEND BLOBS** (DE ADICIÓN) ESTÁN FORMADOS POR BLOQUES, AL IGUAL QUE LOS BLOBS DE BLOQUES, PERO ESTÁN OPTIMIZADOS PARA OPERACIONES DE ADICIÓN. SON IDEALES PARA ESCENARIOS COMO EL REGISTRO DE DATOS DE MÁQUINAS VIRTUALES. SE USAN GENERALMENTE PARA GUARDAR LOGS.

LOS **PAGE BLOBS** (DE PÁGINA) ALMACENAN ARCHIVOS DE ACCESO ALEATORIO DE HASTA 8 TB DE TAMAÑO. SE UTILIZAN PARA ALMACENAR ARCHIVOS DE DISCOS DUROS VIRTUALES (VHD) Y SIRVEN COMO DISCOS PARA MÁQUINAS VIRTUALES DE AZURE.

SISTEMAS DE BUCKETS (AZURE BLOB STORAGE)



Microsoft Azure
Blob Storage



AZURE BLOB STORAGE ESTÁ ORGANIZADO STORAGE STAMPS (SELLOS DE ALMACENAMIENTO) UBICADOS EN VARIOS CENTROS DE DATOS EN TODO EL MUNDO.

CADA STORAGE STAMP CONSTA DE 10 A 20 RACKS, CADA UNO CON ALREDEDOR DE 18 NODOS DE ALMACENAMIENTO (DISCOS + SERVIDORES). EN TOTAL, UN SELLO DE ALMACENAMIENTO PUEDE ALMACENAR HASTA APROXIMADAMENTE 30 PB DE DATOS. UN SELLO DE ALMACENAMIENTO NO SE LLENARÁ A MÁS DEL 80 % DE SU CAPACIDAD TOTAL

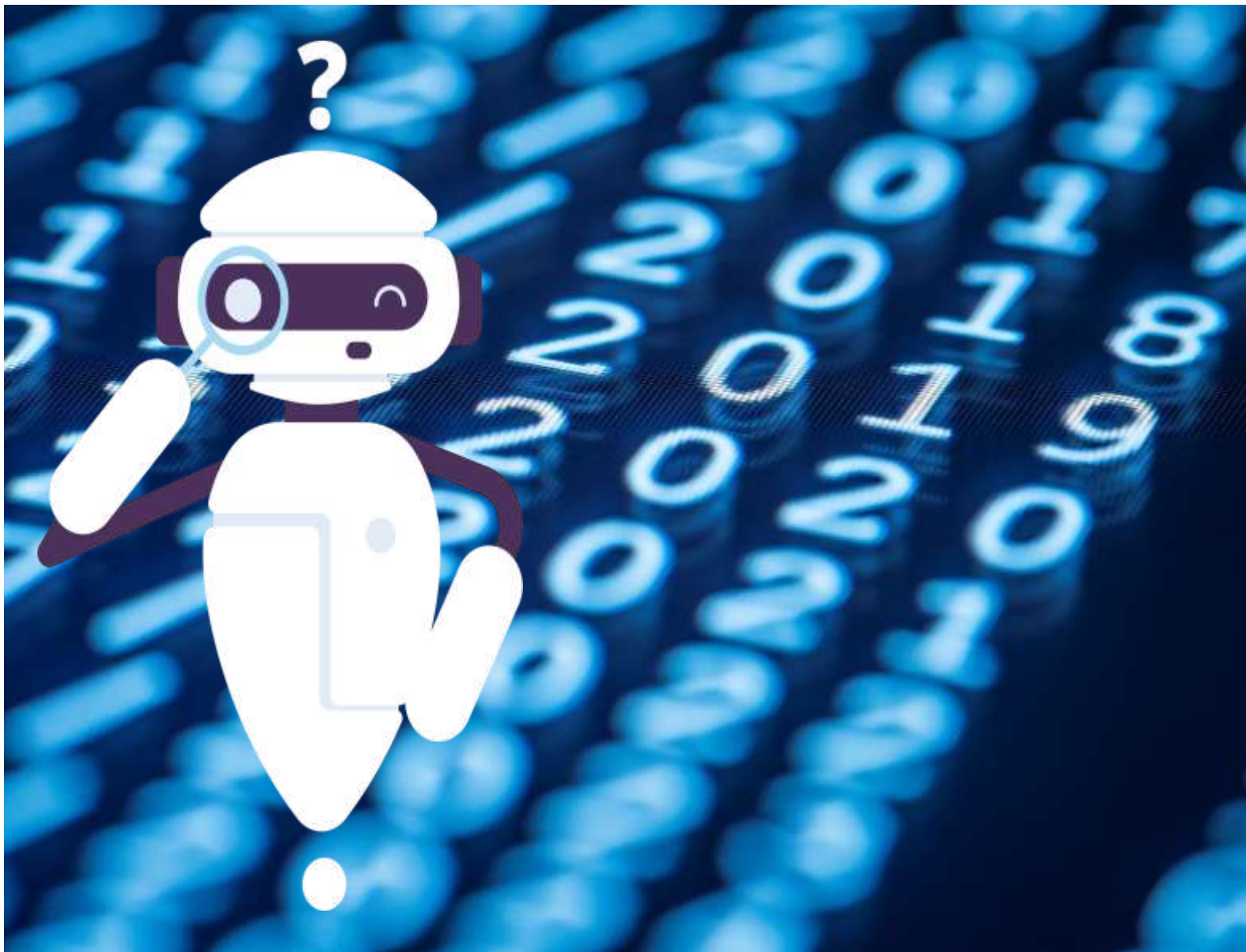
SI UN STORAGE STAMP ALCANZA SU CAPACIDAD MÁXIMA, ALGUNOS DATOS SE REASIGNARÁN A OTRO STORAGE STAMP EN SEGUNDO PLANO. SI NO HAY SUFICIENTES STORAGE STAMPS, SERÁ NECESARIO COMPRAR E INSTALAR NUEVOS RACKS EN LAS UBICACIONES MÁS CONVENIENTES.

POSIBLES PREGUNTAS DE PARCIAL

- ¿Qué es un sistema de archivos distribuido y cómo funciona?
- ¿Por qué los sistemas de archivos distribuidos parecen centralizados para los usuarios?
- ¿Cuáles son las principales ventajas de utilizar un sistema de archivos distribuido?
- ¿Cómo se distribuyen los datos en un sistema de archivos distribuido?
- ¿Cuáles son las características clave de los sistemas de archivos distribuidos?
- ¿Por qué los sistemas de archivos distribuidos deben ser tolerantes a fallos?
- ¿Cómo manejan los sistemas de archivos distribuidos la lectura y escritura de grandes archivos?
- ¿Qué papel juega el paralelismo en los sistemas de archivos distribuidos?
- ¿Cómo se diferencian los sistemas de archivos distribuidos de los sistemas de almacenamiento tradicionales?
- ¿Qué es el teorema CAP y qué significa cada una de sus propiedades?
- ¿Cómo afecta el teorema CAP a los sistemas de archivos distribuidos?
- ¿Qué sucede cuando un sistema distribuido prioriza la consistencia sobre la disponibilidad?
- ¿Qué es la consistencia eventual en un sistema distribuido?
- ¿Cómo se propagan las actualizaciones en un sistema distribuido según el teorema CAP?
- ¿Por qué no se pueden garantizar simultáneamente las tres propiedades del teorema CAP?
- ¿Cómo se maneja la replicación de datos en un sistema distribuido?
- ¿Por qué en un sistema distribuido pueden perderse ciertas restricciones de integridad de datos?
- ¿Qué ocurre si en un sistema distribuido hay una falla parcial en la red?

POSIBLES PREGUNTAS DE PARCIAL

- ¿Qué es Hadoop y cuál es su propósito principal?
- ¿Por qué Hadoop se considera un framework de código abierto para Big Data?
- ¿Cuál fue el problema de Big Data que llevó a la creación de Hadoop?
- ¿Cómo influyó Google en el desarrollo de Hadoop?
- ¿Qué es HDFS y por qué es relevante en el almacenamiento de grandes volúmenes de datos?
- ¿Cómo se organiza la arquitectura de HDFS y cuáles son sus componentes principales?
- ¿Qué función cumple el NameNode en HDFS?
- ¿Cómo se manejan las fallas de los DataNodes en HDFS?
- ¿Qué es el modelo Master-Slave en HDFS y cómo funciona?
- ¿Qué es MapReduce y cómo procesa grandes volúmenes de datos?
- ¿En qué consiste el algoritmo MapReduce y cómo se divide el trabajo en tareas?
- ¿Qué diferencias hay entre un sistema de archivos distribuido y HDFS?
- ¿Por qué HDFS usa replicación de bloques y cuál es el factor de replicación por defecto?
- ¿Qué es un bucket y cómo se diferencia de un sistema de archivos tradicional?
- ¿Cómo se identifican los objetos dentro de un bucket?
- ¿Por qué los sistemas de almacenamiento en buckets son altamente escalables?
- ¿Qué diferencias existen entre AWS S3 y Azure Blob Storage?
- ¿Qué tipos de blobs ofrece Azure Blob Storage y para qué se usan?
- ¿Cómo se organiza el almacenamiento en Azure Blob Storage?
- ¿Qué características de seguridad y redundancia tienen los sistemas de buckets en la nube?



GRACIAS

¿ALGÚN PUNTO QUE
QUIERAN REPASAR O
PROFUNDIZAR?